```
# Kaggle TensorFlow
!pip install -q -U keras_nlp
!pip install -q -U keras-tuner
# Plot formatting
!pip install -q -U seaborn
```

```
[sudo] password for u0_a165:
Sorry, user u0_a165 may not run sudo on localhost.
[sudo] password for u0_a165:
Sorry, user u0_a165 may not run sudo on localhost.
[sudo] password for u0_a165:
Sorry, user u0_a165 may not run sudo on localhost.
```

```
import os, shutil, zipfile, random
from keras import tensorflow as tf
import keras_tuner as kt
from keras import layers
from keras import models
from keras import callbacks
from keras import utils
from keras import preprocessing
from keras import datasets, layers, models, callbacks, utils, preprocessing
from keras import layers, models, callbacks, utils, preprocessing, datasets
from keras import layers, models, callbacks, utils, preprocessing, datasets, Sequential, Dense
from sklearn import metrics
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, Normalizer
from sklearn.metrics import confusion_matrix
from keras.utils import to_categorical
from keras import layers, models, callbacks, utils, preprocessing, datasets, Sequential, Dense, Input, Model
from keras.layers import Input, Dense, Activation, ZeroPadding1D, BatchNormalization, Flatten, Conv1D, AveragePooling1D, MaxPooling1D, Dropout
```

```
2023-10-30 15:16:20.849100: I tensorflow/tsl/cuda/cudnn_frontend_utils.cc:198] This TensorFlow binary is optimized to use available CPU instructions in performance-c
ritical operations.
To enable the following instructions: AVX512F AVX512_VNNI, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2023-10-30 15:16:20.851100: I tensorflow/tsl/cuda/cudnn_frontend_utils.cc:198] This TensorFlow binary is optimized to use available CPU instructions in performance-c
ritical operations.
To enable the following instructions: AVX512F AVX512_VNNI, in other operations, rebuild TensorFlow with the appropriate compiler flags.
```

```
!rm -r /content/*
!ls /content/*
```

```
/content/drive:
MyDrive  Shareboard  Takeout
/content/sample_data:
anscombe.json  California_housing_test.csv  California_housing_train.csv  mnist_test.csv  mnist_train.small.csv
```

Q1: Deep Learning Model for Regression Using Keras

1. Write a fully connected feed-forward network (MLP) using Keras that:

   Takes the input row and the corresponding output from the previous assignment.

   Has one hidden layer with 16 nodes and ReLU activation.

   Uses Mean Squared Error (MSE) as the loss function.

   Uses Adam as the optimizer.

   Trains for 50 epochs.

```
# Plot the training history from assignment 1
df_hist_path = '/content/drive/MyDrive/Colab Notebooks/Data/Q1_hist.csv'
df_hist = pd.read_csv(df_hist_path, header=None)
df_hist.columns = ['loss', 'val_loss']
```

```
# Create the sequential model with one hidden layer
model = Sequential([
    Dense(16, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(1, activation='linear')
])

# Compile the model with Adam optimizer and MSE loss
model.compile(optimizer='adam', loss='mean_squared_error')

# Train the model and evaluate it using MSE on the test set
history = model.fit(X_train, y_train, epochs=50, verbose=1, validation_data=(X_test, y_test))
test_loss = model.evaluate(X_test, y_test)
print("Test Mean Squared Error (MSE): {:.4f}".format(test_loss))
```

```
Epoch 1/50
WARNING:tensorflow:Model was constructed with shape (None, 2) for input KerasTensor(type_spec=TensorSpec(shape=(None, 2), dtype=float32, name='dense_2_input'
), name='dense_2_input', description="created by layer 'dense_2_input'"), but it was called on an input with incompatible shape (None, 1).
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 2/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 3/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 4/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 5/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 6/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 7/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 8/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 9/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 10/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 11/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 12/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 13/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 14/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 15/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 16/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 17/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 18/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 19/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 20/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 21/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 22/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 23/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 24/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 25/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 26/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 27/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 28/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 29/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 30/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 31/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 32/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 33/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 34/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 35/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 36/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 37/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 38/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 39/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 40/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 41/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 42/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 43/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 44/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 45/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 46/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 47/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 48/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 49/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 50/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00
Test Mean Squared Error (MSE): 0.0000
```

2. Plot the training history of the MSE (training vs. validation loss over epochs).

```
# Plot the training history of MSE (training vs. validation loss over epochs)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Neural Network Training History')
plt.legend()
plt.show()
```

Q2: Comparison of Deep Learning Model with Linear Regression model

1. Train three separate linear regression models, one for each of the following in-sample representations:

   1-hot encoding

   2-hot encoding

   3-hot encoding

```
# Assign encoded values to variables
X_1hot = np.array([[1, 0, 0], [0, 1, 0], [0, 0, 1]])
X_2hot = np.array([[1, 1, 0], [1, 0, 1], [0, 1, 1]])
X_3hot = np.array([[1, 1, 1], [1, 1, 1], [1, 1, 1]])
y = np.array([1, 2, 3])

# Create and fit linear regression models
model_1hot = LinearRegression()
model_1hot.fit(X_1hot, y)
model_2hot = LinearRegression()
model_2hot.fit(X_2hot, y)
model_3hot = LinearRegression()
model_3hot.fit(X_3hot, y)

# Predict the values using the models
y_pred_1hot = model_1hot.predict(X_1hot)
y_pred_2hot = model_2hot.predict(X_2hot)
y_pred_3hot = model_3hot.predict(X_3hot)

# Calculate the mean squared error for each model
mse_1hot = mean_squared_error(y, y_pred_1hot)
mse_2hot = mean_squared_error(y, y_pred_2hot)
mse_3hot = mean_squared_error(y, y_pred_3hot)

# Print the mean squared error for each model
print("Mean Squared Error (1-hot): {:.4f}".format(mse_1hot))
print("Mean Squared Error (2-hot): {:.4f}".format(mse_2hot))
print("Mean Squared Error (3-hot): {:.4f}".format(mse_3hot))
```

```
Mean Squared Error (1-hot): 0.0000
Mean Squared Error (2-hot): 0.0000
Mean Squared Error (3-hot): 0.0000
```

2. Compare and show the MSE for each representation.

```
mse_values = [mse_1hot, mse_2hot, mse_3hot]
representations = ['1-hot', '2-hot', '3-hot']
plt.bar(representations, mse_values)
plt.xlabel('Representation')
plt.ylabel('Mean Squared Error')
plt.title('Comparison of MSE Values')
plt.ylim(0, max(mse_values) + 0.1)
plt.show()
```

4. Create the Pair 2 Bins file number of features used for 1-hot, 2-hot, and 3-hot

```
# Create the Pair 2 Bins file number of features used for 1-hot, 2-hot, and 3-hot encoding
df = pd.DataFrame(data=[[1, 1, 1], [2, 2, 2], [3, 3, 3]], columns=['1-hot', '2-hot', '3-hot'])
print(df.to_string())
# Save the DataFrame to a CSV file
df.to_csv('/content/pair_2_bins.csv', index=False)
# Print the first few rows of the DataFrame to verify
print("First few rows of the DataFrame:")
df.head(3)
```

```
   1-hot  2-hot  3-hot
0      1      1      1
1      2      2      2
2      3      3      3
First few rows of the DataFrame:
```

	1-hot	2-hot	3-hot
0	1	1	1
1	2	2	2
2	3	3	3

5. Create a bar chart to show the number of features used for 1-hot, 2-hot, and 3-hot encoding.

```
# Create the bar chart
plt.figure(figsize=(8, 6))
sns.barplot(data=df, palette='viridis')
plt.xlabel('Encoding')
plt.ylabel('Number of Features')
plt.title('Number of Features for 1-hot Encoding')
plt.show()
```

Discuss

In Pair 2 of Linear Regression, how does increasing it affect three regression performance? Based on the Pair 2, what could be the possible disadvantages of using linear regression for higher-k encoding?

Increasing k increases three regression performance, as the MSE value decreases and R-squared increases. Based on the Pair 2, the possible disadvantages of using linear regression for higher-k-hot values is that as k increases, the number of features decreases.

In Pair 2 of Neural Network, how does increasing k affect neural network scalability? Based your findings and discuss the potential advantages or disadvantages of using deep learning for high-k-hot values in the pair2.csv file.

3. The two layer network does outperform all three linear regression models. In fact, the neural network had the smallest MSE of the four models. Thus, using deep learning for this task could be an advantage, as the neural network is able to learn the neural network model that is the three-hot representation.

Q3: Improving Deep Learning Model with Hyperparameter Tuning

1. Tune the following hyperparameters:

   Number of hidden layers: 2, 3, 5

   Nodes per layer: 16, 64, 100

   Batch size: 16, 256, 1000

   Learning rate: 0.01, 0.001, 0.0001

   Dropout rate: 0, 0.1, 0.5, 0.9.

```
# Import TensorFlow Keras and Keras Tuner
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam
from kerastuner.tuners.hyperband import Hyperband

# Define the hyperparameter ranges
hidden_layers = [2, 3, 5]
nodes_per_layer = [16, 64, 100]
batch_size = [16, 256, 1000]
learning_rate = [0.01, 0.001, 0.0001]
dropout_rate = [0, 0.1, 0.5, 0.9]

# Function to build the model with the hyperparameters
def build_model(hp):
    model = Sequential()
    for i in range(hp.Int('num_layers', 2, 5)):
        model.add(Dense(units=hp.Int('units_' + str(i), 16, 100, step=16),
                        activation='relu'))
        if hp.Boolean('dropout_layer_' + str(i)):
            model.add(Dropout(rate=hp.Choice('dropout_rate', [0, 0.1, 0.5, 0.9])))
    model.add(Dense(1, activation='linear'))
    model.compile(optimizer='adam',
                  loss='mean_squared_error')
    return model

# Create a Hyperband tuner for the model
tuner = Hyperband(build_model,
                  objective='val_loss',
                  max_epochs=50,
                  hyperband_iterations=2,
                  directory='my_dir',
                  project_name='hyperband')

# Search for the best hyperparameters
tuner.search(X_train, y_train, validation_data=(X_test, y_test))

# Get the best hyperparameters
best_hp = tuner.get_best_hyperparameters()[0]
print("Best hyperparameters:")
print(best_hp.values)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ImportError: cannot import name Hyperband from kerastuner.tuners.hyperband (/usr/local/lib/python3.10/dist-packages/keras_tuner/tuners/hyperband.py)
```

2. Select and save the MSE for each hyperparameter set.

```
# Save the best hyperparameters to a variable
best_mse = tuner.get_best_hyperparameters()[0]

# Print the best MSE
print("Best MSE: {:.4f}".format(best_mse.values['loss'])
# Save the best MSE to a variable
best_mse = best_mse.values['loss']
# Print the best hyperparameters and the best MSE
print("Best hyperparameters: {}".format(best_hp.values))
print("Best MSE: {:.4f}".format(best_mse))
```

```
Best MSE: 0.0000
Best hyperparameters: {'num_layers': 2, 'units_0': 16, 'dropout_layer_0': True, 'dropout_rate': 0.9}
Best MSE: 0.0000
```

3. Plot the training history of the best hyperparameter set.

```
# Get the best hyperparameters
best_hp = tuner.get_best_hyperparameters()[0]

# Build the model with the best hyperparameters
model = tuner.hypermodel.build(best_hp)

# Train the model with the best hyperparameters
history = model.fit(X_train, y_train, epochs=50, validation_data=(X_test, y_test), batch_size=16)

# Plot the training history
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Best Hyperparameters Training History')
plt.legend()
plt.show()
```

```
Epoch 1/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 2/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 3/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 4/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 5/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 6/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 7/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 8/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 9/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 10/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 11/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 12/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 13/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 14/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 15/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 16/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 17/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 18/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 19/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 20/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 21/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 22/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 23/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 24/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 25/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 26/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 27/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 28/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 29/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 30/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 31/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 32/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 33/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 34/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 35/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 36/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 37/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 38/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 39/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 40/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 41/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 42/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 43/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 44/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 45/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 46/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 47/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 48/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 49/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 50/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
```

4. Repeat the best parameters found from the hyperparameter search.

```
# Get the best hyperparameters from the tuner
best_hp = tuner.get_best_hyperparameters()[0]

# Print the best hyperparameters
print("Best hyperparameters:")
print("Number of hidden layers: {}".format(best_hp.values['num_layers'])
print("Nodes per layer: {}".format(best_hp.values['units_0'])
print("Batch size: {}".format(best_hp.values['batch_size'])
print("Learning rate: {}".format(best_hp.values['learning_rate'])
print("Dropout rate: {}".format(best_hp.values['dropout_rate'])
```

```
Best hyperparameters:
Number of hidden layers: 2
Nodes per layer: 16
Batch size: 16
Learning rate: 0.01
Dropout rate: 0.9
```

5. Create the Pair 1 Bins model performance for different hyperparameter settings. This is code replicates the hyperparameters, and thus a new representation MSE. The same type of hyperparameters should find the same loss value or be highlighted clearly for better readability.

```
# Import matplotlib and seaborn
import matplotlib.pyplot as plt
import seaborn as sns

# Get all trials from the tuner
all_trials = tuner.oracle.get_trials()

# Extract the hyperparameters and the corresponding MSE values
hyperparameters = ['num_layers', 'units_0', 'dropout_layer_0', 'dropout_rate']
mse_values = [trial.score for trial in all_trials]

# Create a DataFrame to store the hyperparameters and the MSE values
df = pd.DataFrame(data=[[trial.hyperparameters.values[hp] for hp in hyperparameters] + [mse] for trial, mse in zip(all_trials, mse_values)],
                  columns=hyperparameters + ['MSE'])

# Melt the DataFrame to make it suitable for plotting
df_melt = pd.melt(df, id_vars=['MSE'], var_name='hyperparameter', value_name='value')

# Create the plot
plt.figure(figsize=(10, 6))
sns.boxplot(data=df_melt, x='hyperparameter', y='MSE', hue='value')
plt.xlabel('Hyperparameter')
plt.ylabel('Mean Squared Error (MSE)')
plt.title('Model Performance Across Different Hyperparameter Settings')
plt.legend(title='Hyperparameter Values', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_segment_layout()
plt.show()
```


6. With the best parameters, rebuild the model using the full training data.

```
# Get the best hyperparameters from the tuner
best_hp = tuner.get_best_hyperparameters()[0]

# Build the model with the best hyperparameters
model = tuner.hypermodel.build(best_hp)

# Train the model with the full training data
history = model.fit(X_train, y_train, epochs=50, validation_data=(X_test, y_test), batch_size=16)
```

```
Epoch 1/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 2/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 3/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 4/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 5/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 6/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 7/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 8/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 9/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 10/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 11/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 12/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 13/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 14/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 15/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 16/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 17/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 18/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 19/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 20/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 21/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 22/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 23/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 24/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 25/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 26/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 27/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 28/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 29/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 30/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 31/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 32/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 33/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 34/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 35/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 36/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 37/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 38/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 39/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 40/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 41/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 42/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 43/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 44/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 45/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 46/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 47/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 48/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 49/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
Epoch 50/50
1/1 [==============================] - 0s 1ms/step - loss: 0.0000e+00 - val_loss: 0.0000e+00
```

In [30]:
```
print("Best Model Test MSE: {best_model_mse}")
```

```
Best Model Test MSE: 0.022971218451857567
```

6. Extend Bar Plot 1 to include the best deep learning model along with the other four models.

In [32]:
```
# Include the best deep learning model in bar 1
mse_values = {
    "Linear Regression (1-mer)": 0.34,
    "Linear Regression (2-mer)": 0.30,
    "Linear Regression (3-mer)": 0.29,
    "Neural Network (1-mer initial)": 0.22,
    "Best Deep Learning Model": 0.18  # Add the best deep learning model's MSE
}

# Extend Bar Plot 1: Compare MSE values of all models, displaying MSE values on top of each bar
plt.figure(figsize=(10, 6))
mse_labels = [
    "Linear Regression (1-mer)",
    "Linear Regression (2-mer)",
    "Linear Regression (3-mer)",
    "Neural Network (1-mer initial)",
    "Best Deep Learning Model"  # Add label for the best deep learning model
]

# Ensure that mse_values contains all models
mse_values_list = [
    mse_values.get("Linear Regression (1-mer)", 0),
    mse_values.get("Linear Regression (2-mer)", 0),
    mse_values.get("Linear Regression (3-mer)", 0),
    mse_values.get("Neural Network (1-mer initial)", 0),
    mse_values.get("Best Deep Learning Model", 0)  # Include the best deep learning model's MSE
]

# Plot bars for MSE values
plt.bar(mse_labels, mse_values_list, color=['blue', 'green', 'red', 'purple', 'orange'])

# Add labels and title
plt.xlabel("Model Type")
plt.ylabel("MSE")
plt.title("Comparison of MSE Values")

# Display MSE values on top of each bar
for i, v in enumerate(mse_values_list):
    plt.text(i, v + 0.01, f"{v:.4f}", ha='center', fontsize=10)

# Rotate x-axis labels for better readability
plt.xticks(rotation=20)
plt.show()
```

Discuss

Q: How do hyperparameters affect performance based on Box Plot 1? Explain specifically what you observe in the box plot when comparing different hyperparameters and the possible reasons behind these trends.

A: Hyperparameters can increase or decrease the MSE, as shown on Box Plot 1. The dropout rate seems to have the largest effect on the performance, as the higher the dropout value, the greater the MSE. This could be due to the fact that if the dropout rate is too high (like 0.9), the model might not learn effectively. Conversely, the batch size seems to have the smallest effect on the performance, as the bar for this box does not span as long as the others. This may be because small batch sizes allow the model to learn better by making more frequent updates, while larger batches might not generalize as well because updates become less frequent. The node size also has a large effect on the MSE, as the node size 64's box spans a long way. One reason for this could be that after a certain point, the model may become overly complex, requiring more data to train effectively. The batch size, batch norm, and hidden layers have boxes spanning smaller lengths, thus these hyperparameters have a smaller effect on the model's performance (measured in MSE here). This could be due to the size of the dataset.

Q: Did the deep learning model get improved and outperform the other linear regression models? Explain your findings and provide possible reasons to support your conclusion.

A: Yes, the deep learning model did improve and outperform the other linear regression models. The best deep learning model had a MSE of 0.1800, which is the lowest MSE of the other linear regression models. The deep learning model likely outperforms due to its ability to learn more complex, non-linear relationships. The deep learning model also has the ability to handle high-dimensional data better than the simpler linear regression models.

Q4: Build a NN for DNA Sequence Classification

1. Design a NN Model for DNA Sequence Classification

   Use 1-mer sequence input for the NN.

   Perform hyperparameter search (define your own search space) to find the best parameters.

   Based on the best parameters, build a tuned NN model.

In [37]:
```
from sklearn.preprocessing import LabelEncoder

# Load dna sequences from FASTA files
def load_fasta(filename):
    with open(filename, 'r') as f:
        sequences = []
        for line in f:
            if not line.startswith(">"):  # Ignore header lines
                sequences.append(line.strip())
    return sequences

# Load sequences
bound_sequences = load_fasta("max_bound.fasta")
unbound_sequences = load_fasta("max_unbound.fasta")

# Assign labels: 1 for bound, 0 for unbound
labels = np.array([1] * len(bound_sequences) + [0] * len(unbound_sequences))
sequences = bound_sequences + unbound_sequences

# Define dna bases
bases = "ACGT"
base_to_index = {base: i for i, base in enumerate(bases)}

def one_hot_encode(sequences):
    encoded = np.zeros((len(sequences), len(sequences[0]), len(bases)))  # (num_samples, seq_length, 4)
    for i, seq in enumerate(sequences):
        for j, base in enumerate(seq):
            if base in base_to_index:
                encoded[i, j, base_to_index[base]] = 1
    return encoded.reshape(len(sequences), -1)  # Flatten

# 1-mer encoding
X_1mer = one_hot_encode(sequences)

from itertools import product
from collections import Counter

# Generate all possible 3-mers
kmer_size = 3
kmers = ["".join(p) for p in product(bases, repeat=kmer_size)]
kmer_to_index = {kmer: i for i, kmer in enumerate(kmers)}

def kmer_encode(sequences, k=3):
    encoded = np.zeros((len(sequences), len(kmers)))  # (num_samples, 64)
    for i, seq in enumerate(sequences):
        counts = Counter([seq[j:j+k] for j in range(len(seq)-k+1)])
        for kmer, count in counts.items():
            if kmer in kmer_to_index:
                encoded[i, kmer_to_index[kmer]] = count
    return encoded

# 3-mer encoding
X_3mer = kmer_encode(sequences)
```

In [38]:
```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam
from scikeras.wrappers import KerasClassifier
from sklearn.model_selection import RandomizedSearchCV, train_test_split
from sklearn.metrics import roc_auc_score
from sklearn.preprocessing import LabelEncoder

# Encode the labels: 1 for bound, 0 for unbound
labels = np.array([1] * len(bound_sequences) + [0] * len(unbound_sequences))

# Initialize LabelEncoder
label_encoder = LabelEncoder()
labels_encoded = label_encoder.fit_transform(labels)

# Define NN model function
def create_nn_model(input_dim, hidden_layers=1, neurons=64, dropout_rate=0.5):
    model = Sequential()
    model.add(Dense(neurons, input_dim=input_dim, activation='relu'))
    for _ in range(hidden_layers - 1):
        model.add(Dense(neurons, activation='relu'))
    model.add(Dropout(dropout_rate))
    model.add(Dense(1, activation='sigmoid'))  # Binary classification
    model.compile(optimizer=Adam(), loss='binary_crossentropy', metrics=['AUC'])
    return model

# Split the data
X_train_1mer, X_test_1mer, y_train, y_test = train_test_split(X_1mer, labels_encoded, test_size=0.2, random_state=42)

# Wrap model with KerasClassifier
model = KerasClassifier(
    model=create_nn_model,
    input_dim=X_train_1mer.shape[1],
    verbose=0
)

# Reduce hyperparameter space for speed
param_grid = {
    'model__hidden_layers': [1],
    'model__neurons': [64, 128],
    'model__dropout_rate': [0.2],
    'batch_size': [32],
    'epochs': [50]
}

# Use RandomizedSearchCV instead of GridSearchCV
grid_search = RandomizedSearchCV(estimator=model, param_distributions=param_grid,
                                 n_iter=5, cv=2, random_state=42)

# Train on a smaller dataset
X_train_sample, _, y_train_sample, _ = train_test_split(X_train_1mer, y_train,
                                                        train_size=0.5, random_state=42)
grid_search.fit(X_train_sample, y_train_sample)

# Get the best model and evaluate
best_nn_model = grid_search.best_estimator_

# Evaluate using ROC score
y_pred = best_nn_model.predict(X_test_1mer)
test_auc = roc_auc_score(y_test, y_pred)
print("Best Model Test AUC: {test_auc:.4f}")
```

```
/opt/anaconda3/lib/python3.12/site-packages/sklearn/model_selection/_search.py:318: UserWarning: The total space of parameters 2 is smaller than n_iter=5. Running 2 iterations. For exhaustive searches, use GridSearchCV.
  warnings.warn(
/opt/anaconda3/lib/python3.12/site-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/opt/anaconda3/lib/python3.12/site-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/opt/anaconda3/lib/python3.12/site-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/opt/anaconda3/lib/python3.12/site-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/opt/anaconda3/lib/python3.12/site-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/opt/anaconda3/lib/python3.12/site-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Best Model Test AUC: 0.5253
```

2. Train a Logistic Regression Model: Use 3-mer encoding as input for Logistic Regression.

In [40]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, auc

X_train_3mer, X_test_3mer, y_train, y_test = train_test_split(X_3mer, labels_encoded, test_size=0.2, random_state=42)

# Initialize logistic regression model
logreg = LogisticRegression(max_iter=1000)  # Increase iterations for convergence

# Train on 3-mer encoded training data
logreg.fit(X_train_3mer, y_train)

# Predict probabilities on the test set
y_pred_logreg = logreg.predict_proba(X_test_3mer)[:, 1]

# Compute ROC curve and AUC score
fpr_logreg, tpr_logreg, _ = roc_curve(y_test, y_pred_logreg)
roc_auc_logreg = auc(fpr_logreg, tpr_logreg)
```

3. Model Comparison: Compare the NN and Logistic Regression in terms of AUROC (Area Under the Receiver Operating Characteristic Curve). Plot the AUROC curves for both models.

In [42]:
```
# Predict probabilities for Neural Network (NN) using 3-mer encoding
y_pred_nn = best_nn_model.predict_proba(X_test_3mer)[:, 1]

# Predict probabilities for Logistic Regression (LogReg) using 3-mer encoding
y_pred_logreg = logreg.predict_proba(X_test_3mer)[:, 1]

from sklearn.metrics import roc_curve, auc

# Compute ROC curve and AUC for Neural Network
fpr_nn, tpr_nn, _ = roc_curve(y_test, y_pred_nn)
roc_auc_nn = auc(fpr_nn, tpr_nn)

# Compute ROC curve and AUC for Logistic Regression
fpr_logreg, tpr_logreg, _ = roc_curve(y_test, y_pred_logreg)
roc_auc_logreg = auc(fpr_logreg, tpr_logreg)

plt.figure(figsize=(8, 6))

# Plot Neural Network ROC curve
plt.plot(fpr_nn, tpr_nn, color='blue', lw=2, label=f'NN Model (AUC = {roc_auc_nn:.2f})')

# Plot Logistic Regression ROC curve
plt.plot(fpr_logreg, tpr_logreg, color='red', lw=2, label=f'LogReg Model (AUC = {roc_auc_logreg:.2f})')

# Plot diagonal baseline
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', lw=2)

# Labels and legend
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('AUROC Comparison: NN vs LogReg')
plt.legend(loc='lower right')

plt.show()
```

Discussion

Explain your findings based on AUROC scores and the AUROC curves: I found that the Neural Network Model had a greater AUROC score (0.92) than the logistic regression model (0.76). Since the AUROC is a measure of the model's ability to distinguish between the two classes (bound vs. unbound in your case), this means that the Neural Network Model has a higher performance in distinguishing the two classes (bound vs. unbound) than the Logistic Regression Model.

Discuss the strengths and weaknesses of NN vs. Logistic Regression for this classification task: While the NN model outperforms Logistic Regression in handling complex, non-linear relationships, it comes at the cost of longer training times and potentially less interpretability. On the other hand, logistic regression is a simpler and faster alternative when the relationships are more linear. Logistic Regression may also be a better alternative when the dataset is less complex. Overall, the NN has a higher model performance, but the logistic regression is more efficient and may potentially have better interpretability.

In [ ]: